중도탈락 예측으로 배우는 머신러닝 기초

로지스틱 회귀 · XGBoost

데이터 분석 입문자 과정 | 외국인 유학생 학적 데이터 13,090명 실측 기반

오늘 다루는 것

네 부분으로 나눠 진행합니다

부	주제	핵심 질문
1부	**전처리**	모델이 바뀌면 전처리가 왜 바뀌는가 / 상관관계와 VIF는 무엇이 다른가
2부	**로지스틱 회귀**	계수와 오즈비를 어떻게 읽는가 / 어떤 옵션을 건드려야 하는가
3부	**XGBoost**	부스팅은 어떻게 작동하는가 / 수많은 옵션 중 무엇이 중요한가

오늘의 문제 — 누가 학교를 떠날 위험이 있는가

풀어야 할 문제의 성격

이진 분류 (binary classification) — 0(유지) 또는 1(탈락) 둘 중 하나를 맞히는 문제입니다

정답(타깃)은「제적여부」. 이미 결과가 나온 학생에게서 규칙을 배웁니다

데이터의 구조

학습용 = 졸업·수료·제적한 학생 (결과 확정) → 여기서 규칙을 배웁니다

예측용 = 지금 재학·휴학 중인 학생 (결과 미정) → 여기에 규칙을 적용합니다

이 데이터의 가장 큰 특징 — 불균형

학부생 제적률 10.5%

1부

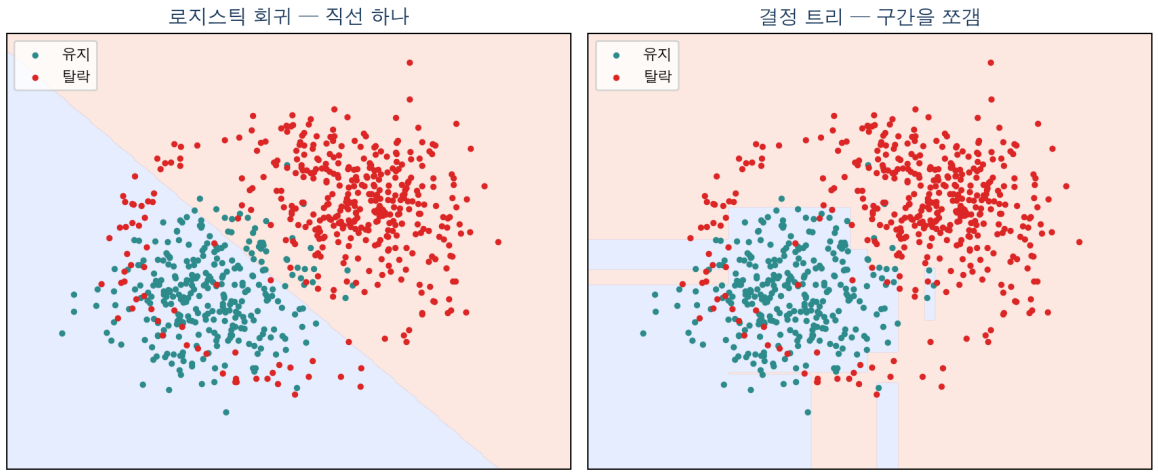
전처리

전처리에 정답은 없습니다. 어떤 모델을 쓸지가 정답을 정합니다.

- 모델이 데이터를 쓰는 방식이 다르면, 데이터를 준비하는 방식도 달라집니다
- 결측치 · 범주형 인코딩 · 스케일링 — 무엇을 왜 하는가
- 상관관계와 VIF — 변수끼리 겹칠 때 무슨 일이 일어나는가
- 실전 사례: 계수 계산이 아예 불가능해진 순간

두 모델은 애초에 다르게 판단합니다

전처리의 모든 차이가 여기서 나옵니다



로지스틱 회귀 — 모든 변수를 하나의 저울에 올려 더합니다

$z = b_0 + b_1 \cdot$이수학기수 $+ b_2 \cdot$최종GPA $+ \cdots$ → 직선(평면,초평면) 하나로 나눕니다

XGBoost(트리) — 질문을 이어 붙여 구간을 쪼갭니다

「최종GPA < 1.5 인가?」→「휴학학기수 ≥ 2 인가?」→ 계단 모양 경계가 만들어집니다

> 더하는 모델은 단위를 맞춰야 하고, 나누는 모델은 순서만 맞으면 됩니다. 이 한 줄이 전처리 차이의 전부입니다.

전처리 대조표 — 무엇이 어떻게 다른가

항목	로지스틱 회귀	XGBoost	왜 다른가
범주형	원핫 더미화 필수	category 타입 그대로	선형식은 숫자만 더할 수 있음
기준 범주	하나를 빼야 함 (drop='first')	불필요	전부 넣으면 합이 1로 고정됨
스케일링	필수	불필요	「1.5보다 작은가」는 단위를 바꿔도 답이 같음
결측치	직접 채워야 함	그대로 둬도 됨	XGBoost는 결측을 어느 쪽으로 보낼지 학습
다중공선성	VIF 점검 필수	거의 무관	계수가 불안정 / 아무거나 쓰면 됨
범주 많은 변수	더미 폭발 → 묶어야 함	최적 분할 탐색	28개 범주 = 27개 열 추가
하이퍼파라미터	사실상 없음	탐색 필요	트리 깊이·개수·학습률을 정해야 함

① 결측치 — 빈칸을 어떻게 메울 것인가

메운 값은 관측된 사실이 아니라 추측입니다. 그래서 결측률을 먼저 봅니다

방법	설명	언제 쓰나	주의
행 삭제	결측이 있는 행을 버림	결측이 아주 적을 때	표본이 줄고 편향이 생길 수 있음
평균 대체	전체 평균으로 채움	분포가 대칭일 때	이상치에 끌려감
중앙값 대체	전체 중앙값으로 채움	이상치가 있을 때 (권장)	분산이 실제보다 작아짐
최빈값 대체	가장 흔한 값으로 채움	범주형 변수	흔한 범주가 더 흔해짐
그룹 평균	같은 학과 평균으로 채움	집단 차이가 클 때 (권장)	그룹 전체가 결측이면 못 채움
결측 표시자	「비어 있었다」를 0/1 변수로 추가	결측 자체가 정보일 때	변수가 늘어남
그대로 두기	알고리즘에 맡김	XGBoost, LightGBM	선형 모델에서는 불가능

> 절대 원칙 ① 결측률 50% 넘으면 기본 제외 (실측: 합격순위 78% 결측 → 제외)

② 범주형 인코딩과「더미변수 함정」

글자로 된 변수를 숫자로 바꾸는 일 — 여기서 초급자가 가장 많이 넘어집니다

원핫 인코딩 (one-hot encoding)

계열 =「공학」인 학생 → 계열_공학=1, 계열_예체능=0, 계열_인문사회=0, 계열_자연과학=0

이렇게 해야「공학이면 얼마를 더한다」는 계산이 가능해집니다

더미변수 함정 (dummy variable trap)

계열이 4종류인데 더미는 3개만 만듭니다. 4개를 다 넣으면 네 열의 합이 항상 1이 됩니다

→ 한 열이 나머지로 완전히 계산되므로 계수를 구하는 방정식에 답이 하나로 정해지지 않습니다

→ sklearn에서는 OneHotEncoder(drop='first'), pandas에서는 get_dummies(drop_first=True)

빠진 하나를「기준 범주」라 하고, 나머지 계수는 전부「기준 대비 차이」로 읽습니다

XGBoost는 astype("category") + enable_categorical=True 로 원본을 그대로 씁니다

※ sklearn은 파이썬에서 머신러닝을 할 때 가장 많이 쓰는 라이브러리 중 하나인 **scikit-learn**의 패키지 이름
※ pandas는 파이썬에서 표 형태의 데이터를 다루는 대표적인 라이브러리

③ 스케일링 — 단위를 맞추는 일

나이(20~40)와 GPA(0~4.5)를 그대로 더하면 저울이 한쪽으로 기웁니다

왜 로지스틱 회귀에는 필요한가

계수 크기를 서로 비교하려면 단위가 같아야 합니다

L2 정규화가 모든 계수에 똑같은 벌점을 매기므로, 단위가 다르면 불공평해집니다

경사하강법 계열 solver의 수렴이 훨씬 빨라집니다 (ConvergenceWarning 예방)

왜 트리 모델에는 불필요한가

트리는「GPA가 1.5보다 작은가?」처럼 순서만 봅니다. 단위를 바꿔도 순서는 그대로입니다

두 가지 방법

StandardScaler — 평균 0, 표준편차 1로. 정규분포에 가까울 때 (기본 선택)

MinMaxScaler — 최소 0, 최대 1로. 범위가 정해진 값일 때. 이상치에 약합니다

순서를 틀리면 누수가 생깁니다

① 학습셋/검증셋 분리 → ② 학습셋으로만 fit → ③ 검증셋에는 transform만

전체 데이터로 fit하면 검증셋의 평균을 미리 엿본 것이 됩니다

StandardScaler 계산식

$$z = \frac{x - \mu}{\sigma}$$

- x: 원래 값
- μ: 평균
- σ: 표준편차
- z: 변환된 값

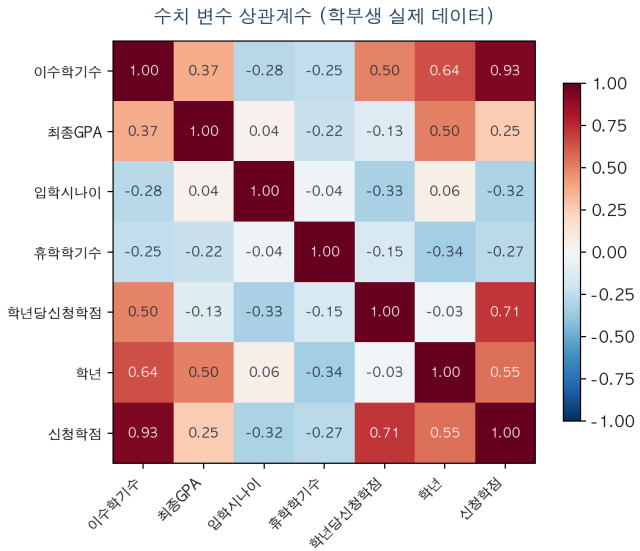
MinMaxScaler 계산식

기본적으로 0~1 범위로 변환할 때는

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

④ 상관관계 — 변수끼리 얼마나 닮았나

두 변수가 함께 움직이는 정도를 -1 ~ +1 사이 숫자로 나타냅니다



읽는 기준

|r| < 0.3 약함 · 0.3~0.7 보통 · 0.7~0.9 강함(주의) · > 0.9 거의 같은 변수

이수학기수 ↔ 신청학점 = 0.93

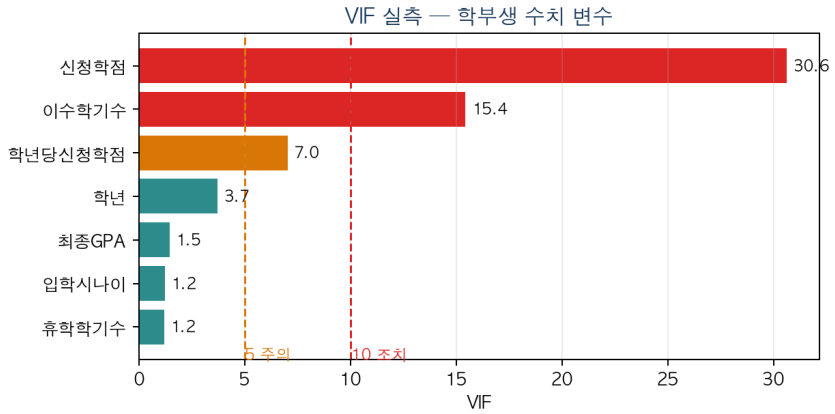
학기를 많이 다닐수록 신청 학점이 쌓이니 당연합니다. 둘 다 넣으면 계수가 불안정해집니다

「신청학점」을 빼고 「학년당신청학점」만 씁니다

> 상관관계의 한계: 한 번에 두 변수씩만 봅니다. A와 B가 각각 C를 만들어내는 경우는 못 잡습니다. → 그래서 VIF가 필요합니다.

⑤ VIF — 여러 변수를 한꺼번에 보는 진단

VIF(분산팽창계수) = 그 변수를 나머지 변수 전부로 얼마나 잘 예측할 수 있는가



$$R^2 = 1 - \frac{SSE}{SST}$$

여기서

- y_i: 실제 값
- $\hat{y}_i$: 회귀모형이 예측한 값
- $\bar{y}$: 실제 값의 평균
- SSE: 예측하고도 남은 오차
- SST: 실제 값이 평균에서 얼마나 퍼져 있는지

$$GPA=\beta_0+\beta_1(\text{취득학점})+\beta_2(\text{학사경고})+\beta_3(\text{휴학횟수})+\varepsilon$$

여기에 수식을 입력하십시오.VIF = 1 / (1 − R²)　← R²은「그 변수를 나머지 변수들로 회귀했을 때의 설명력」

1 = 전혀 안 겹침 · 5 이상 주의 · 10 이상 조치 필요 · ∞ = 완전히 겹침(계산 불가)

상관관계와 무엇이 다른가

상관관계는 두 변수씩. VIF는 나머지 전부와 한꺼번에 — 훨씬 강한 진단입니다

반드시 더미화·스케일링이 끝난 설계행렬에서 계산합니다. 원본에서 계산하면 의미가 없습니다

> 더미 변수의 높은 VIF는 수치 변수만큼 심각하지 않습니다. 기준 범주를 무엇으로 잡느냐에 따라 값이 달라지기 때문입니다.

실전 사례 — 계수 계산이 아예 불가능해진 순간

① 증상 — VIF = ∞

학부 모델에 「계열」과 「소속 대학」을 둘 다 넣었더니 VIF가 무한대인 변수가 2개 나왔다면

② 원인 추적 — 교차표로 확인

예술대학 → 전원 예체능 · 문과대학 → 전원 인문사회 · 사회과학대학 → 전원 인문사회

소속 대학 12개 중 10개가 계열 하나로만 매핑 → 「계열」은 「소속 대학」 더미들의 합으로 완벽히 재현됨

③ 결과 — statsmodels가 멈춤

LinAlgError: Singular matrix ← 역행렬이 존재하지 않아 계수를 구할 수 없습니다

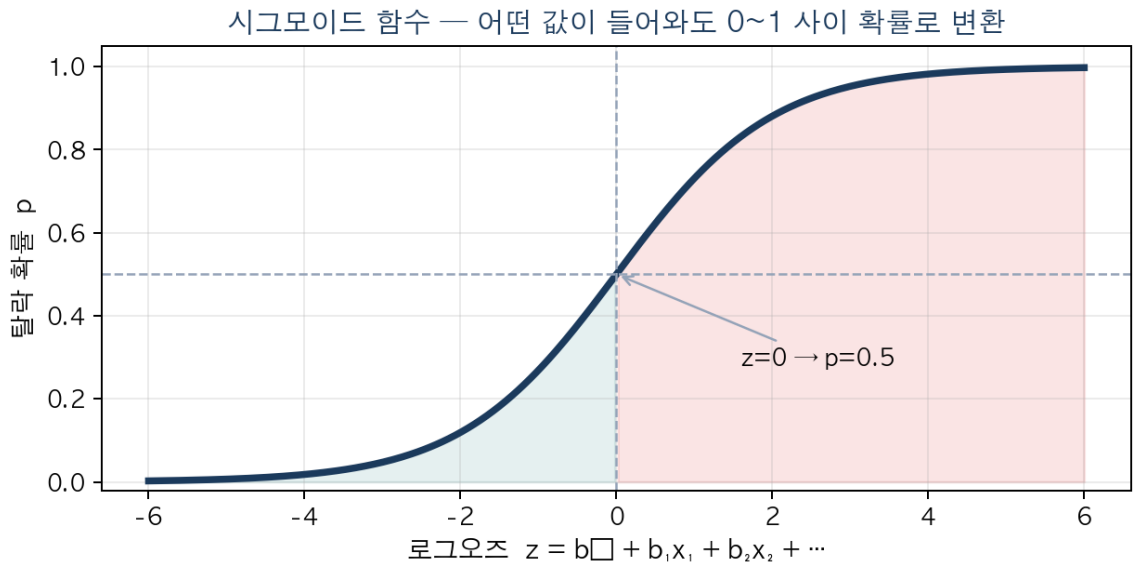
$$\beta = (X^T X)^{-1} X^T y$$

④ 해결 — 둘 중 하나를 뺍니다

1. VIF가 높은 변수 확인
2. 어떤 변수와 중복되는지 상관계수와 변수 정의 확인
3. 중복 변수라면 하나 제거
4. 비슷한 의미의 변수라면 통합 또는 파생변수 재설계
5. 연구적으로 꼭 필요한 변수라면 유지할 수도 있음
6. 처리 후 VIF를 다시 계산

로지스틱 회귀는 어떻게 확률을 만드나

선형식의 결과를 시그모이드 함수에 통과시켜 0~1 사이 확률로 바꿉니다



왜 일반 선형회귀를 쓰면 안 되나

선형회귀는 −3.2나 1.7 같은 값을 내놓습니다. 확률은 0과 1 사이여야 하는데 그 보장이 없습니다

세 단계로 이해하기

① 선형식으로 점수를 계산 (로그오즈) → ② 시그모이드로 0~1 확률 변환 → ③ 임계값으로 0/1 판정

학습이란 정답을 가장 잘 맞히는 b_0, b_1, b_2…를 찾는 과정입니다 (최대우도추정)

왜 로지스틱 회귀는 확률 → 오즈 → 로그오즈로 바꿀까?

핵심: 이진 결과의 확률을 "비교 가능한 비율"로 만든 뒤, 선형식과 연결 가능한 범위로 변환한다.

확률 p

$$p$$

범위: 0 ~ 1

사건이 발생할 확률
예: 중도탈락 확률 0.8

문제: 선형식은 음수·양수 모두 가능

비율화 ❯

오즈 Odds

$$p / (1 - p)$$

범위: 0 ~ ∞

발생 가능성 ÷ 미발생 가능성
예: 0.8 / 0.2 = 4

→ "발생이 미발생보다 4배"

로그 ❯

로그오즈 Logit

$$\log[p / (1 - p)]$$

범위: −∞ ~ +∞

오즈에 로그를 취함
0.5 확률 → odds 1 → log 0

→ 선형식과 같은 범위 확보

회귀 ❯

선형식 연결

$$\log[p/(1-p)] = \beta_0 + \beta_1 X_1 + \cdots$$

선형식과 연결

각 변수의 계수 β를 학습
β의 부호 → 방향
exp(β) → 오즈비(OR), 배수 해석

왜 "오즈"를 쓰는가?

① 이진 결과에 자연스러움

발생 가능성과 미발생 가능성을 직접 비교
(p=0.8 → odds=4)

② 기준점이 명확함

p=0.5이면 odds=1
로그오즈는 0이 되어 중심점이 깔끔

③ 영향력을 배수로 해석

회귀계수 β를 exp(β)로 바꾸면
"오즈가 몇 배 변하는가"로 설명 가능

한 줄 정리 | 오즈는 "발생 vs 미발생"을 비교하기 위해, 로그오즈는 그 값을 선형식(−∞~+∞)과 연결하기 위해 사용한다.

오즈비 읽는 법 — 가장 많이 틀리는 대목

로지스틱 회귀를 쓰는 가장 큰 이유가 바로 이 해석 가능성입니다

오즈(odds) = 일어날 확률 ÷ 안 일어날 확률

제적 확률이 20%라면 오즈 = 0.2 / 0.8 = 0.25

$$\text{log-odds} = \beta_0 + 1.099X \qquad e^{1.099} = 3$$

X가 1증가할때 회귀계수가 1.099일때 중도탈락 오즈가 3배가 된다.

오즈비(odds ratio) = 오즈끼리의 비율 = exp(계수)

OR > 1 → 위험을 올림 · OR = 1 → 영향 없음 · OR < 1 → 위험을 낮춤

흔한 오해 세 가지

오즈 = 0.2 / 0.8 = 0.25, 오즈비 3배 = 0.25*3 = 0.75

$$p = \frac{\text{odds}}{1 + \text{odds}}$$

① 「오즈비 3.0 = 위험이 3배」 → 아닙니다. 오즈가 3배입니다. 사건이 드물 때만 비슷해집니다

② 스케일링했는데 원단위로 해석 → 「1 표준편차 늘 때」입니다. 「1학기 늘 때」가 아닙니다

③ 범주형 계수를 절대값으로 해석 → 「기준 범주 대비 차이」입니다

$$p = \frac{0.75}{1 + 0.75} = \frac{0.75}{1.75} \approx 0.4286$$

실무 팁 — 도구를 나눠 쓰세요

sklearn = 예측·임계값·교차검증 / statsmodels = 계수·p값·신뢰구간 (정규화 없음)

> 계수 부호가 도메인 상식과 반대로 나오면 모델이 아니라 다중공선성과 데이터 누수를 먼저 의심하세요.

오즈비(OR) 예시: 계수 1.099는 왜 오즈 3배인가?

이수학기수의 회귀계수 β = 1.099인 경우

회귀계수 β = 1.099 → **OR = exp(1.099) ≈ 3** → **이수학기수 1 증가 → 중도탈락 오즈 3배**

① 원래 확률

중도탈락 확률

p_0 = 20% = 0.20

→

② 오즈 계산

$odds_0 = 0.20 / 0.80$

= 0.25

→

③ OR=3 적용

$odds_1 = 0.25 \times 3$

= 0.75

→

④ 다시 확률로

$p_1 = 0.75 / (1 + 0.75)$

≈ 0.429 = 42.9%

중요

- 오즈가 3배라는 뜻이지, 확률이 3배라는 뜻은 아님
- 20% × 3 = 60%가 아니라 20% → 42.9%

만약 계수가 β = 3이라면

OR = exp(3) ≈ 20.09

→ 오즈가 약 20배

회귀계수 β → exp(β) → 오즈비(OR) → '오즈가 몇 배 변하는가'로 해석

LogisticRegression 주요 옵션

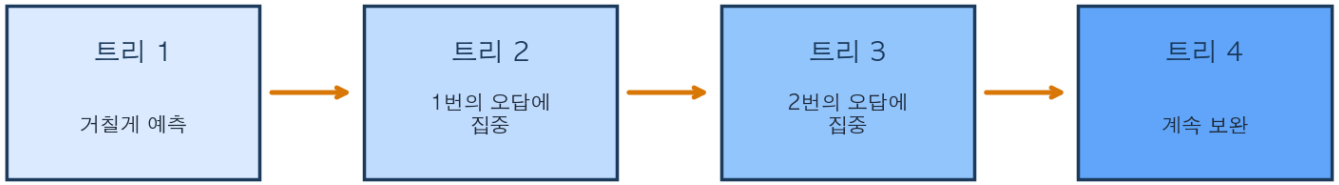
건드릴 것이 많지 않습니다. 그래서 초심자에게 안전한 모델입니다

옵션	기본값	뜻	실무 권장
penalty	'l2'	계수가 커지지 않게 거는 벌점	**'l2' 유지 (완전분리 방지에 필수)**
C	1.0	정규화 강도의 역수 — 작을수록 강하게 억제	변수가 많거나 표본이 적으면 0.1~0.5
class_weight	None	소수 클래스 오답에 주는 가중치	**불균형이면 'balanced' 필수**
solver	'lbfgs'	최적화 알고리즘	기본 유지 / L1이 필요하면 'liblinear'
max_iter	100	최대 반복 횟수	**1000~2000 (수렴 경고 예방)**
random_state	None	난수 시드	재현을 위해 반드시 고정

class_weight='balanced'를 켜면 **클래스 수가 불균형할 때, 적은 클래스의 오류를 더 크게 반영하도록 자동으로 가중치를 주는 옵션**
max_iter는 모델에게 "최적점을 찾을 때까지 최대 몇 번 시도할 수 있니?"라고 정해주는 횟수로 변수 수가 많거나, 데이터가 많을 때 늘릴 필요가 생길 수 있음.

부스팅 — 앞사람이 틀린 것을 다음 사람이 고친다

단순한 트리 수백 개를 이어 붙여 강한 모델을 만듭니다



최종 예측 = 모든 트리의 합 (n_estimators = 트리 개수, learning_rate = 한 번에 고치는 폭)

핵심 아이디어

첫 트리를 만들고 → 그 트리가 틀린 부분에 집중하는 두 번째 트리를 만들고 → 계속 반복합니다

트리 하나는 얕고 약합니다(max_depth 2~6). 약한 것을 여러 개 쌓아 강하게 만듭니다

랜덤 포레스트와의 차이

랜덤 포레스트 = 독립된 트리를 동시에 만들어 평균 / 부스팅 = 순차적으로 앞의 오차를 보완

> 부스팅은 오차를 계속 줄여가기 때문에 내버려두면 학습 데이터를 통째로 외웁니다(과적합). 다음 세 장의 옵션들은 대부분 이것을 막기 위한 장치입니다.

주요 옵션 ① — 트리를 얼마나 복잡하게 만들 것인가

과적합과 과소적합 사이에서 균형을 잡는 옵션들입니다

옵션	기본값	뜻	크게 하면	권장 범위
max_depth	6	트리 한 그루의 최대 깊이	복잡한 관계를 배우지만 외우기 시작함	**2 ~ 6 (표본 적으면 2~4)**
min_child_weight	1	가지를 더 칠 수 있는 최소 표본 무게	잔가지를 덜 쳐서 과적합 억제	**1 ~ 20**
gamma	0	가지를 칠 때 요구하는 최소 개선량	보수적으로 변해 트리가 단순해짐	0 ~ 5
max_leaves	0	잎 개수 제한 (0=제한 없음)	depth 대신 쓰는 방식	grow_policy='lossguide'일 때
max_cat_to_onehot	4	이 수 이하 범주는 원핫, 초과는 최적 분할	원핫으로 처리되는 범주가 늘어남	4 ~ 10

표본이 적을수록(제적 200명 수준) max_depth를 낮게 잡으세요. 깊은 트리는 적은 데이터에서 바로 외워버립니다.

주요 옵션 ② — 얼마나, 얼마나 빨리 배울 것인가

n_estimators와 learning_rate는 반드시 짝으로 생각해야 합니다

옵션	기본값	뜻	실무 권장
n_estimators	100	트리 개수 (부스팅 반복 횟수)	**150 ~ 700 / learning_rate와 반비례로**
learning_rate	0.3	한 트리가 고치는 폭 (eta)	**0.02 ~ 0.2 / 작을수록 트리를 늘려야 함**
subsample	1.0	트리마다 쓸 행(학생)의 비율	0.6 ~ 1.0 / 1보다 작으면 과적합 억제
colsample_bytree	1.0	트리마다 쓸 열(변수)의 비율	0.6 ~ 1.0

> learning_rate를 절반으로 줄이면 n_estimators를 대략 두 배로 늘려야 비슷한 수준에 도달합니다. 「작은 학습률 + 많은 트리」가 대체로 더 좋지만 그만큼 느립니다.

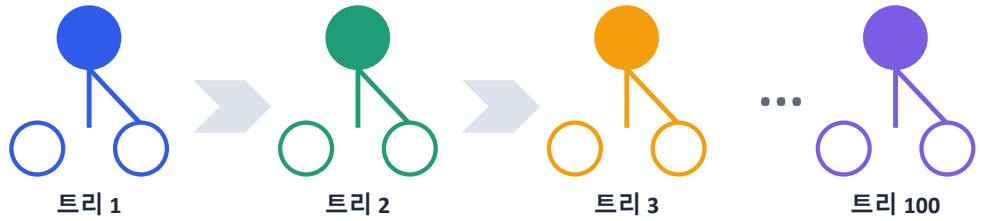
XGBoost 주요 파라미터 ①

트리를 얼마나 만들고, 새 트리의 의견을 얼마나 반영할까?

n_estimators = 트리 개수

예: 100

"작은 의사결정트리를 몇 개 이어서 만들 것인가?"



트리 1이 먼저 예측하고 → **트리 2**가 틀린 부분을 보완 → … → **트리 100**까지 반복

n_estimators ↑ → 더 많이 보완하지만, 너무 많으면 과적합·학습시간 증가 가능

learning_rate = 학습 반영 정도

예: 0.05

"새 트리의 수정 의견을 몇 %만 반영할까?"

learning_rate = 0.3 — 크게 수정



빠르게 접근, 너무 크면 거칠게 학습

learning_rate = 0.05 — 조금씩 수정



천천히 안정적으로 접근 → 보통 트리를 더 많이 사용

기억: learning_rate ↓ ↔ n_estimators ↑ 를 함께 조정하는 경우가 많음

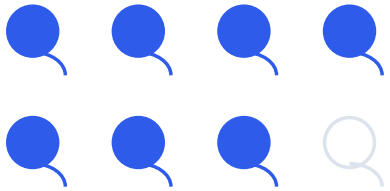
한 줄 요약 | n_estimators = "몇 번 보완?" · learning_rate = "한 번에 얼마나 보완?"

XGBoost 주요 파라미터 ②

무엇을 뽑아 학습하고, 어떤 조건에서 가지를 나눌까?

subsample = 학생(행) 샘플링

예: 0.8



학생 1,000명 중 약 800명만 무작위로 뽑아 한 트리를 학습

→ 같은 학생만 계속 보는 과적합을 줄임

colsample_bytree = 변수 샘플링

예: 0.7



변수 10개 중 약 7개만 골라 트리를 생성

→ 특정 변수 하나에 지나치게 의존하는 것을 줄임

min_child_weight = 작은 가지 제한

예: 5



값이 클수록 작은 자식 노드를 만들기 어려움

※ 정확히는 자식 노드의 Hessian 합 기준

✕ 분할 제한

gamma = "나눌 만큼 이득이 큰가?"

예: 1

분할 전

손실 10.0

분할 후

손실 7.5

개선량 = 2.5

조금 좋아지는 정도라면 가지를 만들지 않음

2.5 > gamma(1) → ✓ 분할 허용

기억하기 | subsample = 학생 뽑기 · colsample = 변수 뽑기 · min_child_weight = 작은 가지 제한 · gamma = 분할 이득 기준

주요 옵션 ③ — 불균형·범주형, 그리고 튜닝 순서

나머지 옵션과, 무엇부터 건드려야 하는가

불균형 데이터 관련

scale_pos_weight — 소수 클래스에 주는 가중치. 보통 (음성 수 / 양성 수) 또는 그 제곱근

max_delta_step — 불균형이 심할 때 한 번에 크게 움직이지 않도록 제한 (1 권장)

- 소수 클래스가 적기 때문에, 특정 잎(leaf)에 소수 클래스 몇 개가 몰리면 그 몇 개의 샘플 때문에 **leaf의 출력값이 크게 튈 수 있습니다.** 이때 예측 확률도 한 번에 과하게 변할 수 있음

eval_metric — 불균형에서는 'aucpr'(PR곡선). 'error'나 'auc'보다 소수 클래스를 잘 반영합니다

범주형·속도 관련

enable_categorical=True + astype('category') — 더미화 없이 범주형을 그대로 사용

튜닝 순서 — 이 순서대로 하세요

① learning_rate를 0.1로 고정하고 n_estimators를 early stopping으로 찾습니다

② max_depth, min_child_weight를 탐색합니다 (영향이 가장 큽니다)

③ subsample, colsample_bytree를 탐색합니다

④ reg_lambda, gamma로 마무리하고, 마지막에 learning_rate를 낮추고 n_estimators를 늘립니다

RandomizedSearchCV